

The Secret Scheme Challenge: Task 4 (Final Task)

[Show/Hide Problem](#)

Submit correct answer to unlock the secret. This is the last task in the challenge, and you need to concatenate answers to all four tasks in sequence to unlock the secret and the final boss.

50612810810978278721

[Unlock](#)

Congratulations on finishing all four tasks! You have earned four stars



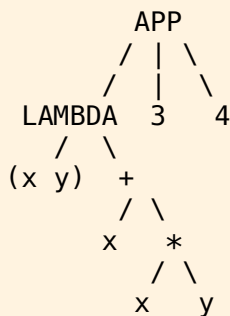
The Secret Life of the Lambdas and the Y-Combinators

We will now take a big and final jump in our interpreter. We now want it to read expressions such as the following.

```
((lambda (x y) (+ x (* x y))) 3 4)
```

Expression Tree

For this expression, the tree looks a bit like:



We will extend our `struct Expr` as follows:

```
struct Expr {
    enum Kind { NUM, VAR, APP, PRIM, LAMBDA, IF } kind;

    int number = 0;           // NUM
    string name;              // PRIM or VAR
    shared_ptr<Expr> func;     // APP
    vector<shared_ptr<Expr>> args; // APP

    // LAMBDA
    string param1, param2;
    shared_ptr<Expr> body;

    // IF
    shared_ptr<Expr> test, thenBranch, elseBranch;
};
```

Note that we have exactly two parameters for the lambda. This is so that we can do something non-trivial using them and at the same time keep our interpreter simple. We explain the earlier example with respect to this structure now.

- `((lambda (x y) (+ x (* x y))) 3 4)` is an application expression of Kind APP.

- `(lambda (x y) (+ x (* x y)))` is a lambda expression of Kind LAMBDA.
 - `x` and `y` just after the string `lambda` are params in Kind LAMBDA.
 - `(+ x (* x y))` is the body field of Kind LAMBDA.
 - `+` is a primitive operation name stored in field name of Kind PRIM.
 - `x` inside `(+ x (* x y))` is a variable reference of Kind VAR.
 - `(* x y)` is an application expression of Kind APP.
 - `*` is a primitive operation name stored in field name of Kind PRIM.
 - `x` inside `(* x y)` is a variable reference of Kind VAR.
 - `y` inside `(* x y)` is a variable reference of Kind VAR.
- `3` is a numeric literal of Kind NUM.
- `4` is a numeric literal of Kind NUM.

We will achieve the application of a lambda by substituting the parameters by the evaluated arguments in the body. E.g., `x` and `y` here will be substituted by `3` and `4`, respectively. This substitution happens recursively and is done in the function called `substitute()`.

Your Task

Your job is to look for `TOD0:s` in the given code to complete it so that lambda expressions are evaluated correctly. **Note:** the parameters can themselves be lambdas! Make sure that you test this as well.

Final Boss: it's not over! There is more after the code below.

Full Code

```
#include <cctype>
#include <iostream>
#include <memory>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

// --- AST ---
struct Expr {
    enum Kind { NUM, VAR, APP, PRIM, LAMBDA, IF } kind;

    int number = 0;           // NUM
    string name;              // PRIM or VAR
    shared_ptr<Expr> func;     // APP
    vector<shared_ptr<Expr>> args; // APP

    // LAMBDA
    string param1, param2;
    shared_ptr<Expr> body;

    // IF
    shared_ptr<Expr> test, thenBranch, elseBranch;
};

// --- Constructors ---
shared_ptr<Expr> num(int n) {
    auto e = make_shared<Expr>();
    e->kind = Expr::NUM;
    e->number = n;
    return e;
}

shared_ptr<Expr> var(const string &s) {
    auto e = make_shared<Expr>();
    e->kind = Expr::VAR;
```

```

    e->name = s;
    return e;
}

shared_ptr<Expr> prim(const string &s) {
    auto e = make_shared<Expr>();
    e->kind = Expr::PRIM;
    e->name = s;
    return e;
}

shared_ptr<Expr> lambda(const string &p1, const string &p2, shared_ptr<Expr> body) {
    // TODO: your code here
}

shared_ptr<Expr> app(shared_ptr<Expr> f, vector<shared_ptr<Expr>> args) {
    auto e = make_shared<Expr>();
    e->kind = Expr::APP;
    e->func = f;
    e->args = args;
    return e;
}

shared_ptr<Expr> iff(shared_ptr<Expr> t, shared_ptr<Expr> th, shared_ptr<Expr> el) {
    auto e = make_shared<Expr>();
    e->kind = Expr::IF;
    e->test = t;
    e->thenBranch = th;
    e->elseBranch = el;
    return e;
}

shared_ptr<Expr> substitute(shared_ptr<Expr> body, const string &p1, shared_ptr<Expr>
                           const string &p2, shared_ptr<Expr> v2) {
    // Note: nested lambdas such as ((lambda (f g) ((lambda (x y) (+ x y)) f g)) 10 1
    // need not be supported.
    // Another example: ((lambda (f g) (+ ((lambda (x y) (+ x y)) f g) 4)) 10 11)
    // I.e., case Expr::LAMBDA: does not need to be handled below.
    // Also, for simplicity, assume that no variable names will be reused.

    switch (body->kind) {
        // TODO: your code here
    }

    throw runtime_error("bad expr");
}

// --- Evaluation ---
shared_ptr<Expr> eval(shared_ptr<Expr> expr) {
    switch (expr->kind) {
        case Expr::NUM:
            return expr;

        case Expr::VAR:
            throw runtime_error("Unbound variable: " + expr->name);

        case Expr::LAMBDA:
            return expr;

        case Expr::PRIM:
            return expr;

        case Expr::IF: {
            auto cond = eval(expr->test);
            if (cond->kind != Expr::NUM)
                throw runtime_error("Condition must be a number");
        }
    }
}

```

```

        if (cond->number != 0)
            return eval(expr->thenBranch);
        else
            return eval(expr->elseBranch);
    }

    case Expr::APP: {
        // primitives
        if (expr->func->kind == Expr::PRIM) {
            string op = expr->func->name;
            // Evaluate first argument as initial value
            int result = eval(expr->args[0])>number;

            // Apply operation across remaining arguments
            for (size_t i = 1; i < expr->args.size(); ++i) {
                int value = eval(expr->args[i])>number;

                if (op == "+")
                    result += value;
                else if (op == "-")
                    result -= value;
                else if (op == "*")
                    result *= value;
                else if (op == "/")
                    result /= value;
                else if (op == "<")
                    result = result < value;
                else
                    throw runtime_error("Unknown operator: " + op);
            }

            return num(result);
        }

        // lambda
        if (expr->args.size() != 2)
            throw runtime_error("Expected 2 arguments");

        // TODO: your code here

        throw runtime_error("Trying to call non-function");
    }
}

throw runtime_error("Unknown expression in eval");
}

// --- Tokenizer ---
vector<string> tokenize(const string &input) {
    vector<string> tokens;
    string current;

    for (char c : input) {
        if (isspace(c)) {
            if (!current.empty()) {
                tokens.push_back(current);
                current.clear();
            }
        } else if (c == '(' || c == ')') {
            if (!current.empty()) {
                tokens.push_back(current);
                current.clear();
            }
            tokens.push_back(string(1, c));
        } else {
            current += c;
        }
    }
}

```

```

    }

    if (!current.empty())
        tokens.push_back(current);

    return tokens;
}

// --- Parser ---
shared_ptr<Expr> parse_expr(vector<string> &tokens, int &pos) {
    if (pos >= tokens.size())
        throw runtime_error("Unexpected end of input");

    string tok = tokens[pos++];

    // primitive
    if (tok == "*" || tok == "/" || tok == "+" || tok == "-" || tok == "<")
        return prim(tok);

    // --- List ---
    if (tok == "(") {
        if (pos >= tokens.size())
            throw runtime_error("Unexpected end after '('");

        string head = tokens[pos++];

        // --- IF ---
        if (head == "if") {
            auto t = parse_expr(tokens, pos);
            auto th = parse_expr(tokens, pos);
            auto el = parse_expr(tokens, pos);

            if (tokens[pos++] != ")")
                throw runtime_error("Expected ')' after if");

            return iff(t, th, el);
        }

        // --- LAMBDA ---
        if (head == "lambda") {
            if (tokens[pos++] != "(")
                throw runtime_error("Expected '(' after lambda");

            string p1 = tokens[pos++];
            string p2 = tokens[pos++];

            if (tokens[pos++] != ")")
                throw runtime_error("Expected ')' after parameters");

            auto body = parse_expr(tokens, pos);

            if (tokens[pos++] != ")")
                throw runtime_error("Expected ')' after lambda body");

            return lambda(p1, p2, body);
        }

        // --- APPLICATION ---
        pos--;
        auto func = parse_expr(tokens, pos);

        vector<shared_ptr<Expr>> args;
        while (pos < tokens.size() && tokens[pos] != ")") {
            args.push_back(parse_expr(tokens, pos));
        }
        pos++; // consume ')'
    }
}

```

```

        return app(func, args);
    }

    // number
    try {
        return num(stoi(tok));
    } catch (...) {
    }

    // variable
    return var(tok);
}

// --- REPL ---
int main() {
    string line;

    cout << "Micro Lisp: Expressions, ifs (numbers only), and lambda(2 args only). "
           "Type 'exit' to quit.\n";

    while (true) {
        cout << ">> ";
        if (!getline(cin, line))
            break;

        if (line == "exit")
            break;
        if (line.empty())
            continue;

        try {
            auto tokens = tokenize(line);
            int pos = 0;
            auto expr = parse_expr(tokens, pos);

            auto result = eval(expr);

            if (result->kind == Expr::NUM)
                cout << result->number << "\n";
            else
                cout << "<function>\n";

        } catch (exception &e) {
            cout << "Error: " << e.what() << "\n";
        }
    }
}

```

The Secret Scheme: A Y-Combinator to the Rescue!

- Submit a lambda expression that is accepted by your interpreter above, following the rules below. The expression will be converted into a canonical string and verified using SHA256.
- Variable names must appear in order starting from **a**, **b**, **c**, **d**, ... based on first use.
- No variable name reuse is allowed.
- The expression must compute fib(10) using exactly the following definition:
 - $\text{fib}(n) = n$ if $n < 2$
 - $\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$ otherwise
- Recursion can be implemented via self-application using a Y-combinator. See [this Stack Overflow answer](#) to read up on Y-combinators, specifically, [this answer](#) has clear examples.
- In addition to the above constraints, the expression should be as short as possible. **Whitespace does not matter.** Before verification, all whitespace is stripped, so formatting will not affect correctness or hashing.

((lambda (a b) (a a b)) (l

Submit

Congratulations! That is the correct answer; here is your fifth star!

